

Universidade Federal do Ceará Campus Quixadá

QXD0043 – Sistemas Distribuídos

Prof. Antônio Rafael Braga

Luna Ayla Souza Bezerra

Trabalho 2:

Relatório

Quixadá-CE
2026

1. Objetivo

O objetivo deste trabalho é adaptar o sistema de loja virtual desenvolvido no Trabalho 1 para utilizar Java RMI na comunicação entre cliente e servidor, substituindo o uso direto de sockets TCP e UDP.

A aplicação continua funcionando em modo texto e permite consultar produtos, registrar pedidos e enviar notificações para vários clientes conectados ao servidor.

2. Alterações em Relação ao Trabalho 1

No Trabalho 1, a comunicação entre cliente e servidor era realizada por sockets TCP, enquanto as notificações eram enviadas por multicast UDP.

No Trabalho 2, essas operações passaram a utilizar Java RMI. O cliente acessa os serviços por meio de uma referência remota, permitindo executar operações no servidor sem precisar implementar sockets manualmente.

O sistema de notificações também foi adaptado. Antes, os clientes entravam e saíam de um grupo multicast utilizando *joinGroup* e *leaveGroup*. Agora, cada cliente registra um objeto remoto no servidor para receber avisos. Quando uma notificação é enviada, o servidor chama o método *receberAviso* dos clientes inscritos.

Dessa forma, vários clientes podem receber a mesma mensagem.

3. Organização das Classes

O sistema possui quatro entidades principais:

- ***Produto:*** representa os produtos disponíveis na loja.
- ***Pedido:*** armazena as informações dos pedidos realizados.
- ***Categoria:*** representa a categoria de um produto.
- ***ClienteLoja:*** representa um cliente e seus pedidos.

Foram implementadas duas relações de herança, nas quais *ProdutoFisico* e *ProdutoDigital* estendem a classe *Produto.*

Também existem duas relações de agregação: *Produto* possui uma *Categoria*, e *ClienteLoja* mantém uma lista de *Pedido.*

As classes *ProdutoService* e PedidoService são responsáveis pelas operações locais realizadas no servidor.

Para a comunicação remota, são utilizadas principalmente as classes *LojaRemota, LojaServidorRemoto, AvisoRemoto, OuvinteAvisos, Comunicacao, Mensagem e JsonSimples.*

4. Funcionalidades Remotas

A loja disponibiliza as seguintes operações:

- ***autenticar:*** verifica as credenciais do usuário.
- ***listarProdutos:*** retorna os produtos disponíveis.
- ***consultarProduto:*** busca um produto pelo identificador.
- ***registrarPedido:*** registra um novo pedido no servidor.
- ***consultarTotalPedido:*** retorna a quantidade de pedidos realizados.
- ***enviarAviso:*** envia uma notificação aos clientes inscritos.

Essas operações são identificadas pelo campo *methodId* e processadas pelo método remoto *executar.*

O sistema também possui os métodos remotos *entrar* e *sair*, utilizados para registrar e remover os clientes que recebem notificações.

5. Protocolo de Requisição-Resposta

A comunicação segue o modelo de requisição e resposta.

A classe *Comunicacao* utiliza o método *doOperation* para preparar a requisição, enviá-la ao servidor por RMI e receber a resposta.

No servidor, *LojaServidorRemoto* utiliza *getRequest* para interpretar a mensagem recebida, identifica a operação solicitada e utiliza *sendReply* para preparar a resposta.

As mensagens são representadas em JSON e possuem os seguintes campos:

- ***messageType:*** identifica requisição (0) ou resposta (1).
- ***requestId:*** identifica a requisição e sua resposta.
- ***objectReference:*** identifica o serviço remoto.
- ***methodId:*** indica a operação solicitada.
- ***arguments:*** contém os argumentos da operação em JSON.

As mensagens são codificadas em UTF-8 e transmitidas como byte[]. O servidor mantém o mesmo *requestId* na resposta, permitindo ao cliente verificar a correspondência entre as mensagens

6. Passagem de Parâmetros

A aplicação utiliza passagem por referência e passagem por valor.

- **Passagem por referência:** o cliente obtém uma referência remota da loja por meio do registro RMI. Além disso, ao se inscrever para receber notificações, fornece ao servidor uma referência remota do seu objeto *AvisoRemoto*. Assim, o servidor consegue chamar *receberAviso* diretamente no cliente.
- **Passagem por valor:** ao registrar um pedido, o cliente envia seus dados em JSON. O servidor recebe essas informações, reconstrói um objeto *Pedido* localmente e realiza o registro. Os resultados das operações também são retornados em JSON.

7. Testes Realizados

O código foi compilado utilizando *javac* e testado com a classe *TesteRMI*.

Durante os testes, o servidor foi iniciado na porta 1099 e foram verificadas as funcionalidades de listagem e consulta de produtos, registro de pedidos, consulta do total de pedidos e envio de notificações.

Também foram utilizados dois ouvintes remotos para verificar se ambos recebiam as mensagens enviadas pelo servidor. Os testes foram concluídos com sucesso.

8. Execução do Sistema

O projeto pode ser executado no Visual Studio Code com a extensão *Extension Pack for Java*.

Primeiro, deve-se executar a classe *Servidor.java* e mantê-la ativa. Em seguida, executa-se a classe *Cliente.java* para acessar a loja. É possível abrir outras instâncias do cliente para testar o recebimento simultâneo de notificações.

Para os testes, foram configurados dois usuários:

Usuário	Senha	Acesso
cliente	123	Consulta produtos e registra pedidos.
admin	admin123	Consulta produtos, visualiza pedidos e envia notificações.

Para testar os avisos, basta manter mais de um cliente conectado e enviar uma mensagem pelo menu do administrador.

9. Limitações

Os produtos e pedidos são armazenados em memória. Por isso, os pedidos registrados são perdidos quando o servidor é reiniciado.

A autenticação utiliza credenciais demonstrativas e não possui um controle completo de sessões e permissões no serviço remoto.

A classe *JsonSimples* implementa apenas o tratamento de JSON necessário para as mensagens utilizadas na aplicação.

Apesar de o Java RMI utilizar comunicação de rede internamente, a aplicação não realiza a criação manual de sockets.

10. Conclusão

O trabalho permitiu adaptar a comunicação da loja virtual para Java RMI, mantendo as funcionalidades principais do sistema e substituindo a implementação direta de sockets.

Com o RMI, foi possível realizar operações remotas, transmitir informações por valor e utilizar referências remotas para enviar notificações aos clientes conectados.

Os testes demonstraram o funcionamento da comunicação entre cliente e servidor e do envio de notificações para múltiplos clientes.